

UNIT – 3 (PYTHON COMPLEX DATA TYPES)

Python Complex data types: Using string data type and string operations, Defining list and list slicing, Use of Tuple data type. String, List and Dictionary, Manipulations Building blocks of python programs, string manipulation methods, List manipulation. Dictionary manipulation, Programming using string, list and dictionary in-built functions. Python Functions, Organizing python codes using functions.

1. STRING DATA TYPE

Definition:

A **string** is a sequence of characters enclosed in single (' '), double (" "), or triple quotes (' ' ' ' or " " " ").

Syntax

```
str1 = 'Hello'
str2 = "Python"
str3 = '''This is
a multiline string.'''
```

Examples:

```
name = "Gautam"
college = 'IET Lucknow'
quote = """Python is easy to learn"""
```

```
print(name)
print(college)
print(quote)
```

Output:

```
Gautam
IET Lucknow
Python is easy to learn
```

String Indexing:

Each character in a string has an **index number** (starting from 0)

```
text = "Python"
print(text[0])    # P
print(text[3])    # h
```

Negative indexing (from end):

```
print(text[-1])   # n
print(text[-3])   # h
```

String Slicing:

We can extract a part of the string using:

```
string[start : end : step]
```

Examples:

```
word = "Engineering"
print(word[0:5])    # 'Engin'
print(word[3:])     # 'ineering'
print(word[:4])     # 'Engi'
print(word[-5:])    # 'neering'
print(word[::-2])   # 'Egnern'
```

✳ Common String Operations:

Operation	Description	Example
+	Concatenate strings	'Data' + 'Science' → 'DataScience'
*	Repeat string	'Hi'*3 → 'HiHiHi'
len()	Find length	len('Python') → 6
in	Membership check	'Py' in 'Python' → True
upper()	Convert to uppercase	'python'.upper()
lower()	Convert to lowercase	'HELLO'.lower()
replace()	Replace part of string	'data'.replace('d','D') → 'Data'
split()	Split into list	'a,b,c'.split(',') → ['a','b','c']
join()	Join list to string	','.join(['a','b','c']) → 'a,b,c'

Common Mistake:

```
s = "hello"
s[0] = 'H' # ❌ Error: Strings are immutable

✅ Correct way:
s = 'H' + s[1:]
```

2. LIST DATA TYPE

Definition:

A **list** is an ordered, mutable collection of items (can be of different data types).

Syntax:

```
list_name = [item1, item2, item3, ...]
```

Example:

```
numbers = [10, 20, 30, 40]
mixed = [10, "Python", 2.5, True]
print(numbers)
print(mixed)
```

Accessing & Slicing:

```
marks = [85, 90, 95, 80, 70]
print(marks[0])    # 85
print(marks[-1])   # 70
print(marks[1:4])  # [90, 95, 80]
```

List Operations:

Operation	Example	Output
Append item	<code>marks.append(100)</code>	<code>[85, 90, 95, 80, 70, 100]</code>
Insert item	<code>marks.insert(1, 88)</code>	<code>[85, 88, 90, 95, 80, 70]</code>
Remove item	<code>marks.remove(90)</code>	removes first 90
Pop item	<code>marks.pop()</code>	removes last element
Sort	<code>marks.sort()</code>	ascending order
Reverse	<code>marks.reverse()</code>	reverses order

⚙️ List Comprehension (shortcut method)

```
squares = [x**2 for x in range(5)]  
print(squares)  
# Output: [0, 1, 4, 9, 16]
```

⚠️ Common Mistake:

```
a = b = [1,2,3]  
a.append(4)  
print(b) # b also changes
```

✅ Correct way:

```
a = [1,2,3]  
b = a.copy()
```

3. TUPLE DATA TYPE

Definition:

A **tuple** is similar to a list but **immutable** (cannot be changed after creation).

Syntax:

```
tuple_name = (item1, item2, item3)
```

Example:

```
t1 = (10, 20, 30)  
print(t1)  
print(t1[1]) # 20
```

Tuple Operations:

Operation	Example	Output
Indexing	t1[0]	10
Slicing	t1[1:]	(20, 30)
Concatenation	(1,2)+(3,4)	(1,2,3,4)
Repetition	(1,2)*2	(1,2,1,2)
Length	len(t1)	3

⚠️ Cannot modify elements:

```
t1[0] = 50 # ❌ Error
```

4. DICTIONARY DATA TYPE

Definition:

A **dictionary** stores data as key-value pairs.

Syntax:

```
dict_name = {key1:value1, key2:value2}
```

Example:

```
student = {"name": "Gautam", "age": 20, "course": "Data Science"}
print(student["name"]) # Gautam
```

⚙️ Dictionary Methods:

Method	Description	Example
keys()	all keys	student.keys()
values()	all values	student.values()
items()	key-value pairs	student.items()
update()	add/modify	student.update({"age": 21})
pop()	remove key	student.pop("age")

🧩 Real-Life Example:

```
# Storing student data
student = {
    "Name": "Gautam",
    "Roll": 21,
    "Marks": [85, 90, 92],
    "Average": sum([85, 90, 92]) / 3
}
print(student)
```

5. BUILDING BLOCKS OF PYTHON PROGRAMS

These are the core parts every program uses:

Building Block	Description	Example
Variables	Store values	x = 10

Data Types	Type of data	int, float, str, list, tuple, dict
Operators	Perform operations	+ - * / %
Input/Output	Interact with user	input(), print()
Control Statements	Make decisions	if, else, for, while
Functions	Reusable blocks	def add(a,b): return a+b

Example Program (Combining All):

```
name = input("Enter student name: ")
marks = [85, 90, 95]
avg = sum(marks)/len(marks)

student = {"name": name, "marks": marks, "average": avg}

print("Student Record:")
print("Name:", student["name"])
print("Average Marks:", student["average"])
```

1. STRING MANIPULATION METHODS

String manipulation means **performing operations** on strings — like changing case, searching, replacing, trimming, etc.

Common String Methods

Method	Description	Example	Output
upper()	Converts to uppercase	'hello'.upper()	'HELLO'
lower()	Converts to lowercase	'PyTHON'.lower()	'python'
title()	First letter of each word capital	'data science'.title()	'Data Science'
capitalize()	First letter capital	'hello'.capitalize()	'Hello'
strip()	Removes spaces	' hello '.strip()	'hello'
replace(a, b)	Replaces substring	'data'.replace('d','D')	'Data'
find(sub)	Returns index of substring	'python'.find('th')	2
count(sub)	Counts substring	'banana'.count('a')	3
split()	Splits into list	'a,b,c'.split(',')	['a', 'b', 'c']

<code>join(list)</code>	Joins list to string	<code>'-'.join(['a','b','c'])</code>	<code>'a-b-c'</code>
<code>startswith(sub)</code>	Checks start	<code>'python'.startswith('py')</code>	True
<code>endswith(sub)</code>	Checks end	<code>'data.csv'.endswith('.csv')</code>	True

Example Program:

```

text = "  I love Python Programming  "

# Basic cleaning
text = text.strip()
print(text.upper())

# Count occurrences
print("Count of 'o':", text.count('o'))

# Replace word
print(text.replace("Python", "Data Science"))

```

◇ 2. LIST MANIPULATION

Important List Methods

Method	Description	Example	Output
<code>append(x)</code>	Adds item at end	<code>a=[1,2]; a.append(3)</code>	<code>[1,2,3]</code>
<code>insert(i,x)</code>	Adds item at position	<code>a.insert(1,5)</code>	<code>[1,5,2,3]</code>
<code>remove(x)</code>	Removes first occurrence	<code>a.remove(2)</code>	<code>[1,5,3]</code>
<code>pop(i)</code>	Removes element at index	<code>a.pop(1)</code>	<code>[1,3]</code>
<code>sort()</code>	Sorts list	<code>a.sort()</code>	<code>[1,3]</code>
<code>reverse()</code>	Reverses list	<code>a.reverse()</code>	<code>[3,1]</code>
<code>count(x)</code>	Count occurrence	<code>[1,2,2].count(2)</code>	2
<code>index(x)</code>	Returns index	<code>[10,20,30].index(20)</code>	1
<code>extend(list2)</code>	Add all from list2	<code>[1,2].extend([3,4])</code>	<code>[1,2,3,4]</code>

Example Program:

```
fruits = ['apple', 'banana', 'mango']
fruits.append('grapes')
fruits.insert(1, 'orange')
fruits.remove('banana')
fruits.sort()
print(fruits)
```

Output:

```
['apple', 'grapes', 'mango', 'orange']
```

3. DICTIONARY MANIPULATION

Common Dictionary Methods

Method	Description	Example	Output
keys()	Returns all keys	d.keys()	dict_keys(['name', 'age'])
values()	Returns all values	d.values()	dict_values(['GKM', 20])
items()	Returns key-value pairs	d.items()	[('name', 'GKM'), ('age', 20)]
get(key)	Returns value safely	d.get('name')	'GKM'
update({...})	Adds or updates	d.update({'course': 'Python'})	Updated dict
pop(key)	Removes key	d.pop('age')	Removes 'age'
clear()	Removes all	d.clear()	{}

Example Program:

```
student = {"name": "Gautam", "age": 20, "course": "Python"}
student.update({"age": 21})
student["grade"] = "A"
print(student.keys())
print(student.values())
print(student)
```

Output:


```
dict_keys(['name', 'age', 'course', 'grade'])
dict_values(['Gautam', 21, 'Python', 'A'])
{'name': 'Gautam', 'age': 21, 'course': 'Python', 'grade': 'A'}
```

◇ 4. PROGRAMMING USING STRING, LIST, & DICTIONARY FUNCTIONS

Example 1: String + List

```
sentence = "python is fun to learn"
words = sentence.split()
print(words)
print("Total words:", len(words))
```

Output:

```
['python', 'is', 'fun', 'to', 'learn']
Total words: 5
```

Example 2: List + Dictionary

```
students = ["Gautam", "Saurabh", "Neha"]
marks = [85, 90, 88]

data = dict(zip(students, marks))
print(data)
```

Output:

```
{'Gautam': 85, 'Saurabh': 90, 'Neha': 88}
```

Example 3: Small Data Processing

```
text = "apple banana apple mango banana apple"
words = text.split()
count_dict = {}

for word in words:
    count_dict[word] = count_dict.get(word, 0) + 1

print(count_dict)
```

Output:

```
{'apple': 3, 'banana': 2, 'mango': 1}
```

5. PYTHON FUNCTIONS

Definition:

A **function** is a block of code that performs a specific task.
It helps **organize**, **reuse**, and **simplify** programs.

Syntax:

```
def function_name(parameters):  
    # function body  
    return value
```

Example:

```
def greet(name):  
    print("Hello", name)  
  
greet("Gautam")
```

Output:

Hello Gautam

Return Example:

```
def add(a, b):  
    return a + b  
  
result = add(5, 10)  
print(result)
```

Output:

15

Types of Functions:

Type	Example
Built-in	len(), type(), sum(), max(), min()
User-defined	Made using def keyword
Lambda (Anonymous)	lambda x : x*x

Example: Lambda Function

```
square = lambda x: x*x  
print(square(5)) # Output: 25
```

Default Parameters:

```
def greet(name="User"):  
    print("Hello", name)
```

```
greet()  
greet("GKM")
```

Output:

```
Hello User  
Hello GKM
```

◇ 6. ORGANIZING PYTHON CODE USING FUNCTIONS

Organizing code using functions makes it:

- Reusable
- Easier to debug
- Structured (like modular programming)

Example Program:

```
def get_average(marks):  
    return sum(marks) / len(marks)  
  
def display_student(name, marks):  
    avg = get_average(marks)  
    print("Student:", name)  
    print("Average Marks:", avg)  
  
# Main program  
student_name = "Gautam"  
marks = [85, 90, 92]  
display_student(student_name, marks)
```

Output:

```
Student: Gautam  
Average Marks: 89.0
```

Real-Life Example (Data Science Context)

```
# Function to calculate mean, max, and min from dataset
def analyze_data(data):
    return {
        "mean": sum(data)/len(data),
        "max": max(data),
        "min": min(data)
    }

scores = [85, 88, 90, 92, 86]
result = analyze_data(scores)
print(result)

Output: {'mean': 88.2, 'max': 92, 'min': 85}
```

 Summary Table

Concept	Key Idea	Use
String Methods	Modify and analyze text	Cleaning data, splitting words
List Methods	Manage ordered data	Handling arrays, storing items
Dictionary Methods	Key-value management	Storing records
Functions	Reusable code	Modular design, clean programs

1. Difference between String, List, and Tuple

Feature	String	List	Tuple
Definition	A sequence of characters.	A sequence of items (can be different types).	Similar to list but immutable (can't change).
Syntax	"Hello" or 'Hello'	[1, 2, 3]	(1, 2, 3)
Mutable / Immutable	Immutable (can't change after creation)	Mutable (can be changed)	Immutable
Example of change	 "abc"[0] = 'x' → Error	 list1[0] = 5 → Works	 tuple1[0] = 5 → Error
Use case	Text handling	Data storage, modifications	Fixed data, faster access

 2. Difference between List and Dictionary

Feature	List	Dictionary
Definition	Ordered collection of items.	Collection of key-value pairs.

Syntax	[10, 20, 30]	{"name": "GKM", "age": 20}
Access method	By index → list[0]	By key → dict["name"]
Order	Ordered (keeps item order)	Ordered since Python 3.7
Duplicates allowed	Yes	Keys must be unique
Mutable / Immutable	Mutable	Mutable
Use case	Store data in sequence	Store data with labels (key-value form)

⚙️ 3. Difference between Mutable and Immutable Data Types

Feature	Mutable	Immutable
Definition	Can be changed after creation	Cannot be changed once created
Examples	List, Dictionary, Set	String, Tuple, Integer
Change effect	Original data changes	New copy is created
Example	list1.append(5) → same list changes	"Hi" + " there" → creates new string
Use case	When frequent updates needed	When data should remain constant

📋 4. Difference between Function and Built-in Function

Feature	User-defined Function	Built-in Function
Definition	Function created by user using def	Function already provided by Python
Example	def add(a,b): return a+b	len(), sum(), max()
Availability	Only in your code	Available by default
Use case	For custom logic	For common operations

📊 5. Difference between List, Tuple, and Dictionary (Quick Summary)

Feature	List	Tuple	Dictionary
Syntax	[]	()	{ }
Mutable	✅ Yes	❌ No	✅ Yes
Access by	Index	Index	Key
Order maintained	✅ Yes	✅ Yes	✅ Yes
Duplicates allowed	✅ Yes	✅ Yes	❌ No (keys)
Use case	Store ordered data	Store fixed ordered data	Store key-value pairs

Most Important Built-in Python Functions

Function Name	Purpose / Use	Syntax	Example & Output
print()	Displays output on screen	<code>print(value)</code>	<code>print("Hello")</code> → Hello
input()	Takes user input (always as string)	<code>input("Message")</code>	<code>name = input("Enter name: ")</code>
type()	Tells data type of a value	<code>type(variable)</code>	<code>type(5)</code> → <class 'int'>
len()	Returns length (no. of items)	<code>len(sequence)</code>	<code>len("GKM")</code> → 3
int(), float(), str()	Convert data types	<code>int(x)</code> , <code>float(x)</code>	<code>int("5")</code> → 5
range()	Creates a sequence of numbers	<code>range(start, stop, step)</code>	<code>list(range(1,5))</code> → [1,2,3,4]
sum()	Adds all elements of a list/tuple	<code>sum(list)</code>	<code>sum([2,4,6])</code> → 12
max()	Finds the largest element	<code>max(sequence)</code>	<code>max([3,8,2])</code> → 8
min()	Finds the smallest element	<code>min(sequence)</code>	<code>min([3,8,2])</code> → 2
sorted()	Returns sorted list	<code>sorted(sequence)</code>	<code>sorted([3,1,2])</code> → [1,2,3]
abs()	Gives absolute (positive) value	<code>abs(number)</code>	<code>abs(-5)</code> → 5
pow()	Raises power	<code>pow(a,b)</code>	<code>pow(2,3)</code> → 8
round()	Rounds a number	<code>round(number, digits)</code>	<code>round(3.14159, 2)</code> → 3.14
id()	Returns memory location of object	<code>id(object)</code>	<code>id(x)</code> → e.g. 1401234
help()	Shows help/documentation	<code>help(object)</code>	<code>help(print)</code> → info about print
dir()	Lists all methods/attributes of object	<code>dir(object)</code>	<code>dir(str)</code> → list of string methods
enumerate()	Returns index + value pairs from sequence	<code>enumerate(sequence)</code>	<code>for i,v in enumerate(['a','b']): print(i,v)</code> → 0 a, 1 b
zip()	Combines multiple lists	<code>zip(list1, list2)</code>	<code>list(zip([1,2], ['a','b']))</code> → [(1,'a'), (2,'b')]
map()	Applies a function to each item	<code>map(func, sequence)</code>	<code>list(map(str, [1,2,3]))</code> → ['1','2','3']
filter()	Filters elements using condition	<code>filter(func, sequence)</code>	<code>list(filter(lambda x:x>5, [3,6,8]))</code> → [6,8]

any()	True if any element is True	<code>any(sequence)</code>	<code>any([0,1,0]) → True</code>
all()	True if all elements are True	<code>all(sequence)</code>	<code>all([1,1,1]) → True</code>
isinstance()	Checks type of variable	<code>isinstance(x, type)</code>	<code>isinstance(5,int) → True</code>
eval()	Executes a string as Python expression	<code>eval(expression)</code>	<code>eval("5+10") → 15</code>

Common Built-in String Functions

(These are very useful in your “String Manipulation” syllabus part)

Function	Purpose	Example	Output
<code>upper()</code>	Converts to uppercase	<code>"gkm".upper()</code>	"GKM"
<code>lower()</code>	Converts to lowercase	<code>"PYTHON".lower()</code>	"python"
<code>title()</code>	Capitalizes each word	<code>"hello world".title()</code>	"Hello World"
<code>strip()</code>	Removes spaces	<code>" hi ".strip()</code>	"hi"
<code>replace(a,b)</code>	Replace substring	<code>"data".replace('a','@')</code>	"d@t@"
<code>split()</code>	Splits into list	<code>"a,b,c".split(',')</code>	['a','b','c']
<code>join()</code>	Joins list to string	<code>" ".join(['Hi','GKM'])</code>	"Hi GKM"
<code>find()</code>	Finds position of substring	<code>"python".find('t')</code>	2
<code>count()</code>	Counts occurrence	<code>"apple".count('p')</code>	2

Common List Functions

Function	Purpose	Example	Output
<code>append(x)</code>	Add item at end	<code>lst.append(4)</code>	[1,2,3,4]
<code>insert(i,x)</code>	Add item at index	<code>lst.insert(1,9)</code>	[1,9,2,3]
<code>remove(x)</code>	Remove by value	<code>lst.remove(2)</code>	[1,3]
<code>pop(i)</code>	Remove by index	<code>lst.pop(1)</code>	[1,3]
<code>sort()</code>	Sort list	<code>lst.sort()</code>	[1,2,3]
<code>reverse()</code>	Reverse list	<code>lst.reverse()</code>	[3,2,1]

Common Dictionary Functions

Function	Purpose	Example	Output
keys()	Returns all keys	d.keys()	dict_keys(['a', 'b'])
values()	Returns all values	d.values()	dict_values([1, 2])
items()	Returns key-value pairs	d.items()	[('a', 1), ('b', 2)]
get(k)	Returns value of key	d.get('a')	1
update()	Updates dictionary	d.update({'c': 3})	{'a': 1, 'b': 2, 'c': 3}
pop(k)	Removes key-value	d.pop('a')	{'b': 2}

UNIT – 3: Python Complex Data Types (Important Q&A)

◇ 1. What are complex data types in Python?

Complex data types are those that can store **multiple values** or **structured data** together.

Examples:

- **String** → sequence of characters
- **List** → sequence of items (can be of different types)
- **Tuple** → ordered but immutable collection
- **Dictionary** → key-value pairs

◇ 2. What is a String in Python? How is it defined?

A **string** is a sequence of characters enclosed in **single** (' '), **double** (" "), or **triple** ("' ") quotes.

Example:

```
name = "GKM"
message = '''Hello Python'''
```


Strings are **immutable**, meaning you cannot change them once created.

◇ 3. What are common string operations?

Operation	Example	Output
Concatenation	"Data" + "Science"	"DataScience"
Repetition	"Hi"*3	"HiHiHi"
Indexing	"Python"[0]	'P'
Slicing	"Python"[1:4]	'yth'
Length	len("GKM")	3

◇ 4. List some common string methods.

Method	Work	Example	Output
upper()	Converts to uppercase	"gkm".upper()	"GKM"
lower()	Converts to lowercase	"PYTHON".lower()	"python"
replace(a,b)	Replace substring	"data".replace('a', '@')	"d@t@"
split()	Split into list	"a,b,c".split(',')	['a', 'b', 'c']
find()	Finds position	"python".find('t')	2

◇ 5. What is a List in Python? How is it different from Tuple?

- A **list** is an **ordered, mutable** collection of items.
- A **tuple** is an **ordered, immutable** collection.

Example:

```
list1 = [1, 2, 3]
```

```
tuple1 = (1, 2, 3)
```

Feature	List	Tuple
Syntax	[]	()
Mutable	✅ Yes	❌ No
Change allowed	Yes	No
Use case	When modification needed	When data fixed

◇ 6. What is list slicing? Give example.

List slicing is used to **access part of a list**.

Syntax: list[start : end : step]

Example:

```
nums = [10,20,30,40,50,60]
print(nums[1:4])    # [20,30,40]
print(nums[::-1])   # reverse list
```

◇ 7. Write some common list methods.

Method	Description	Example	Output
append(x)	Add item at end	l.append(4)	[1,2,3,4]
insert(i,x)	Add at position	l.insert(1,9)	[1,9,2,3]
remove(x)	Remove item	l.remove(2)	[1,3]
pop(i)	Remove by index	l.pop(1)	[1,3]
sort()	Sort list	l.sort()	[1,2,3]
reverse()	Reverse list	l.reverse()	[3,2,1]

◇ 8. What is a Tuple?

Tuple is an **ordered and immutable** collection of elements.

Syntax:

```
t = (10, 20, 30)
```

Once created, elements **cannot be modified**. Used when you want to store fixed data.

◇ 9. What is a Dictionary? Explain with example.

Dictionary stores data in **key-value** pairs.

Each key must be **unique**.

Example:

```
student = {"name": "GKM", "age": 20, "branch": "CSE"}
print(student["name"]) # Output: GKM
```

◇ 10. Write common dictionary methods.

Method	Description	Example	Output
keys()	Returns all keys	d.keys()	['name', 'age']
values()	Returns all values	d.values()	['GKM', 20]

items()	Returns key-value pairs	d.items()	[('name', 'GKM'), ('age', 20)]
get(k)	Get value by key	d.get('name')	GKM
update()	Update dictionary	d.update({'age': 21})	{'name': 'GKM', 'age': 21}
pop(k)	Remove key-value	d.pop('age')	removes 'age'

◇ 11. What is a Function in Python?

A **function** is a block of reusable code used to perform a specific task. It helps in code organization and reusability.

Syntax:

```
def function_name(parameters):
    # code
    return value
```

Example:

```
def add(a, b):
    return a + b

print(add(5,3))    # 8
```

◇ 12. What are advantages of using functions?

- Reduces code repetition
- Makes program modular and readable
- Easier debugging
- Code reuse in multiple places

◇ 13. What are built-in functions related to string, list, and dictionary?

- **String:** len(), upper(), lower(), replace(), split()
- **List:** append(), insert(), sort(), reverse(), pop()
- **Dictionary:** keys(), values(), items(), get(), update()

◇ 14. Write a program to count vowels in a given string.

```
s = "Data Science"
count = 0
for ch in s.lower():
    if ch in 'aeiou':
        count += 1
print("Vowels:", count)
```

Output: Vowels: 5

◇ 15. Write a program to find the maximum element in a list.

```
lst = [3,8,1,6,9]
print("Maximum:", max(lst))
```

Output: Maximum: 9

◇ 16. What is function return value in Python?

When a function finishes execution, it can **send a value back** using return.

If no return, function returns None.

Example:

```
def square(x):
    return x*x
print(square(5))    # 25
```

◇ 17. How to organize Python code using functions?

- Group related tasks into functions
- Place reusable functions in separate .py files (modules)
- Import them using import keyword

Example:

```
# math_utils.py
def add(a,b):
    return a+b
```

Then use:

```
from math_utils import add
print(add(3,4))
```

◇ 18. What is the difference between local and global variables?

Feature	Local Variable	Global Variable
Defined inside	Function	Outside all functions
Accessible	Only within that function	Anywhere in program
Created when	Function is called	Program starts

Example:

```
x = 10
def func():
    y = 5
    print(x+y) # uses global x
func()
```

◇ 19. Explain the use of built-in function map(), filter(), and zip().

Function	Use	Example	Output
map()	Apply function to all elements	list(map(str, [1,2,3]))	['1', '2', '3']
filter()	Filter items by condition	list(filter(lambda x:x>3, [1,4,2,5]))	[4,5]
zip()	Combine two lists	list(zip([1,2], ['a', 'b']))	[(1, 'a'), (2, 'b')]

◇ 20. Short Notes (Very Short Answer Type)

Term	Meaning
Immutable	Cannot be changed after creation (e.g. string, tuple)
Mutable	Can be changed (e.g. list, dict)
Slicing	Access part of sequence (e.g. [1:4])
Key-Value Pair	Data in dictionary stored as key:value
Function	Block of code that performs a specific task